34. Beamforming Signal Rate Conversion

Aim:

To simulate the sampling rate conversion of a beamformed signal from 61.44 MHz to 30.72 MHz and verify the preservation of phase alignment and beamforming integrity.

Software Used: MATLAB (Simulink)

Objectives

1. Convert the sampling rate from 61.44 MHz to 30.72 MHz using decimation.
2. Analyze the phase alignment before and after conversion.
3. Measure amplitude distortion.
4. Compare the input and output spectra.
5. Verify that beamforming performance is preserved.

Program:

```
clc;
clear;
close all;
%% ============================================================
%  BEAMFORMING SIGNAL RATE CONVERSION
%  Input Sampling Rate  = 61.44 MHz
%  Output Sampling Rate = 30.72 MHz
% ============================================================
%% 1. PARAMETERS
Fs_in  = 61.44e6;       % Input sampling frequency
Fs_out = 30.72e6;       % Output sampling frequency
M = Fs_in/Fs_out;       % Decimation factor = 2
N = 8192;               % Number of samples
t = (0:N-1)/Fs_in;
```

```
fc = 5e6;              % Test signal frequency = 5 MHz
numAnt = 4;            % Number of antenna elements
% Phase shifts for four antenna elements
phase_deg = [0 30 60 90];
phase_rad = deg2rad(phase_deg);
fprintf('Input Sampling Rate  = %.2f MHz\n', Fs_in/1e6);
fprintf('Output Sampling Rate = %.2f MHz\n', Fs_out/1e6);
fprintf('Decimation Factor    = %d\n', M);
%% ========================================================
%  GENERATE MULTI-ANTENNA SIGNALS
% ========================================================
x = zeros(numAnt,N);
for k = 1:numAnt
   x(k,:) = exp(1j*(2*pi*fc*t + phase_rad(k)));
end
%% ========================================================
%  BEAMFORMING
% ========================================================
% Steering weights compensate the phase difference
weights = exp(-1j*phase_rad);
beamformed_in = zeros(1,N);
for k = 1:numAnt
   beamformed_in = beamformed_in + weights(k)*x(k,:);
end
beamformed_in = beamformed_in/numAnt;
%% ========================================================
%  TASK 1: IMPLEMENT RATE CONVERSION
% ========================================================
% Anti-aliasing FIR filter
filterOrder = 64;
% Cutoff must be below new Nyquist frequency
cutoff = 0.45/M;
```

```
h = fir1(filterOrder, cutoff);
% Filter every antenna channel
x_filtered = zeros(size(x));
for k = 1:numAnt
    x_filtered(k,:) = filter(h,1,x(k,:));
end
% Remove FIR filter transient/group delay
delay = filterOrder/2;
x_filtered = x_filtered(:,delay+1:end);
% Decimate by factor 2
x_out = x_filtered(:,1:M:end);
% New time axis
Nout = size(x_out,2);
t_out = (0:Nout-1)/Fs_out;
fprintf('\nRate conversion completed.\n');
fprintf('Input samples  = %d\n',N);
fprintf('Output samples = %d\n',Nout);
%% ========================================================
%  BEAMFORM AFTER RATE CONVERSION
% ========================================================
beamformed_out = zeros(1,Nout);
for k = 1:numAnt
    beamformed_out = beamformed_out + weights(k)*x_out(k,:);
end
beamformed_out = beamformed_out/numAnt;
%% ========================================================
%  TASK 2: ANALYZE PHASE RESPONSE
% ========================================================
% Calculate phase difference relative to antenna 1
phase_before = zeros(1,numAnt);
phase_after  = zeros(1,numAnt);
```

```
for k = 1:numAnt
    phase_before(k) = angle(mean(x(k,:).*conj(x(1,:))));
      phase_after(k) = angle(mean(x_out(k,:).*conj(x_out(1,:))));
end
phase_before_deg = rad2deg(phase_before);
phase_after_deg  = rad2deg(phase_after);

phase_error = phase_after_deg - phase_before_deg;
fprintf('\n-----------------------------------------\n');
fprintf('PHASE ANALYSIS\n');
fprintf('-----------------------------------------\n');
fprintf('Antenna   Before(deg)   After(deg)   Error(deg)\n');
for k = 1:numAnt
    fprintf('%d          %8.3f      %8.3f      %8.5f\n',...
        k,...
        phase_before_deg(k),...
        phase_after_deg(k),...
        phase_error(k));
end
%% =========================================================
%  TASK 3: MEASURE AMPLITUDE DISTORTION
% =========================================================

amp_before = rms(abs(beamformed_in));

amp_after = rms(abs(beamformed_out));

amplitude_distortion_percent = ...
    abs(amp_after-amp_before)/amp_before*100;

fprintf('\n-----------------------------------------\n');
fprintf('AMPLITUDE ANALYSIS\n');
```

```
fprintf('----------------------------------------\n');

fprintf('Amplitude before = %.6f\n',amp_before);
fprintf('Amplitude after  = %.6f\n',amp_after);

fprintf('Amplitude distortion = %.6f %%\n',...
    amplitude_distortion_percent);
%% ============================================================
%  TASK 4: COMPARE SPECTRA
% ============================================================

NFFT = 8192;

X_in = fftshift(fft(beamformed_in,NFFT));

X_out = fftshift(fft(beamformed_out,NFFT));

f_in = (-NFFT/2:NFFT/2-1)*(Fs_in/NFFT)/1e6;

f_out = (-NFFT/2:NFFT/2-1)*(Fs_out/NFFT)/1e6;

X_in_dB = 20*log10(abs(X_in)/max(abs(X_in)));

X_out_dB = 20*log10(abs(X_out)/max(abs(X_out)));

figure;

plot(f_in,X_in_dB,'LineWidth',1.5);
hold on;

plot(f_out,X_out_dB,'--','LineWidth',1.5);
grid on;
```

```
xlabel('Frequency (MHz)');
ylabel('Magnitude (dB)');

title('Spectrum Before and After Rate Conversion');

legend('Before Rate Conversion',...
       'After Rate Conversion');

xlim([-20 20]);



%% ========================================================
%  TASK 5: VERIFY BEAMFORMING INTEGRITY
% ========================================================

% Compare coherent beamforming gain

input_power = mean(abs(x_out(1,:)).^2);

beam_power = mean(abs(beamformed_out).^2);

beamforming_gain = 10*log10(beam_power/input_power);

fprintf('\n-----------------------------------------\n');
fprintf('BEAMFORMING INTEGRITY\n');
fprintf('-----------------------------------------\n');

fprintf('Input antenna power = %.6f\n',input_power);

fprintf('Beamformed power = %.6f\n',beam_power);
```

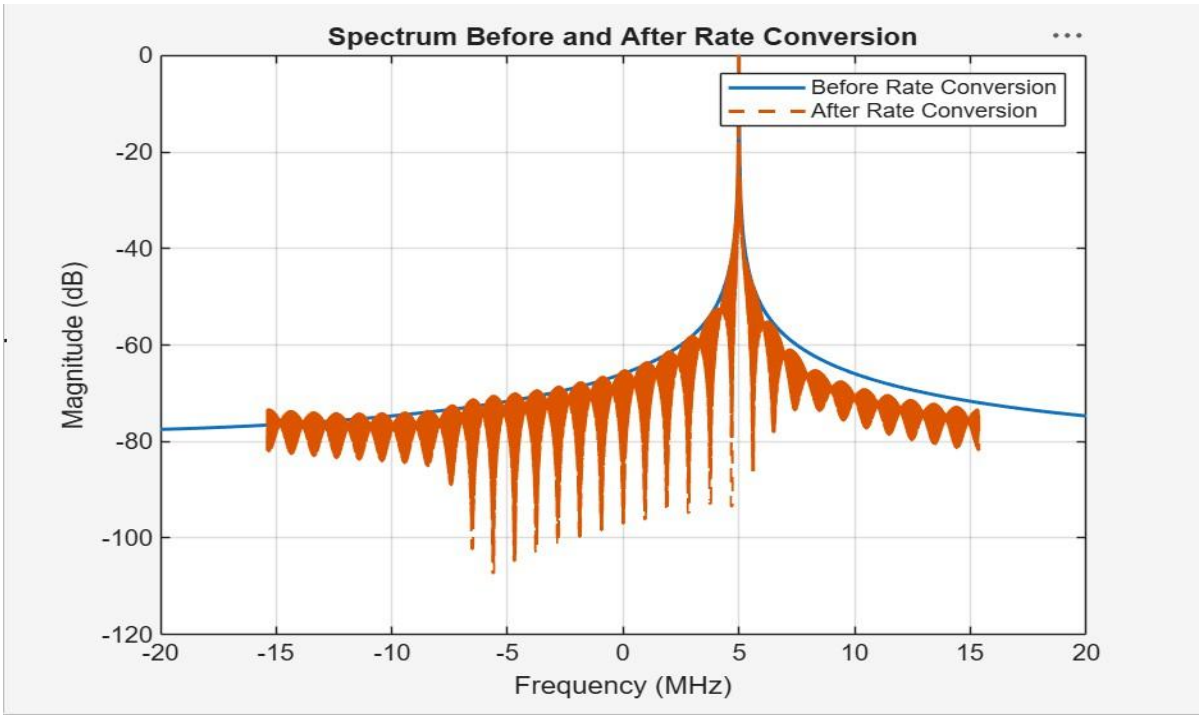
```
fprintf('Beamforming gain = %.3f dB\n',...
    beamforming_gain);


%% =========================================================
%  PHASE COMPARISON GRAPH
% =========================================================

figure;

plot(1:numAnt,...
     phase_before_deg,...
     'o-',...
     'LineWidth',1.5);

hold on;

plot(1:numAnt,...
     phase_after_deg,...
     's--',...
     'LineWidth',1.5);

grid on;

xlabel('Antenna Element');

ylabel('Relative Phase (Degrees)');

title('Phase Alignment Before and After Rate Conversion');

legend('Before Rate Conversion',...
       'After Rate Conversion');
```

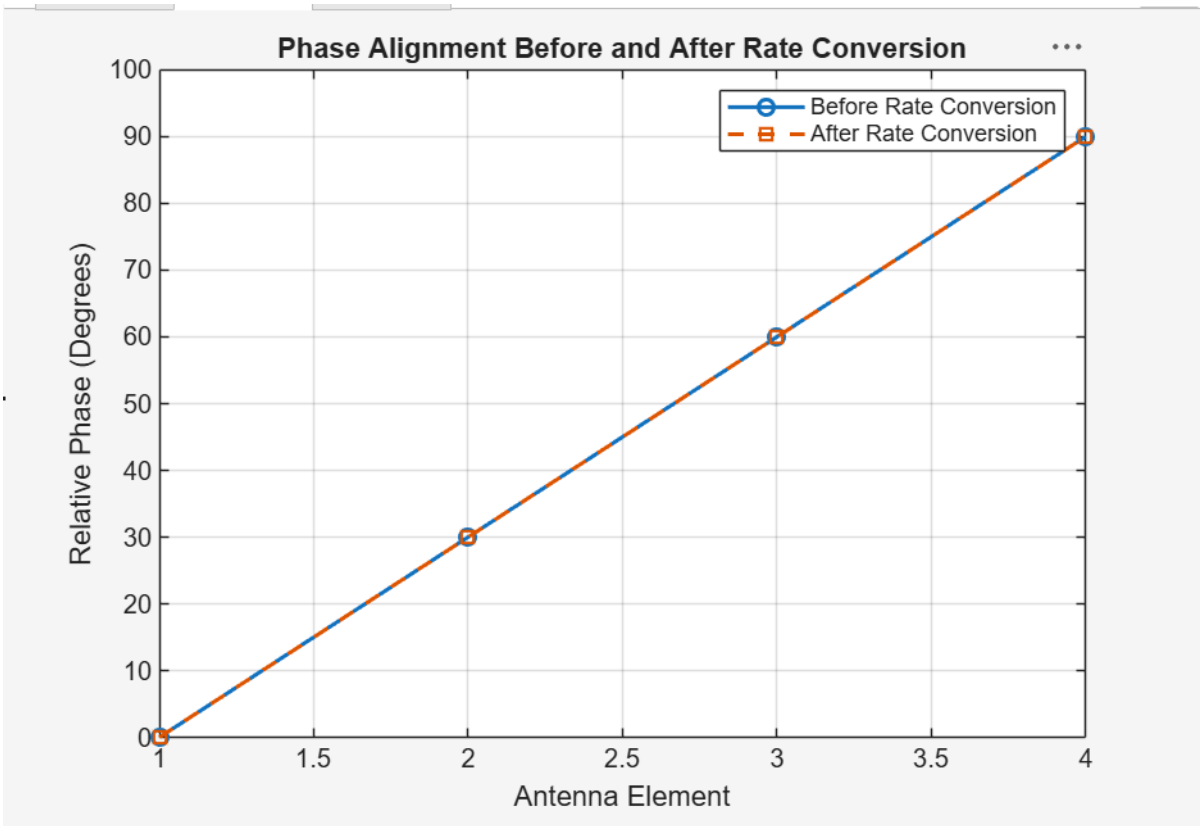
```
%% ========================================================
%  TIME DOMAIN COMPARISON
% ========================================================

figure;

subplot(2,1,1);

plot(t(1:300)*1e6,...
    real(beamformed_in(1:300)));

grid on;

xlabel('Time (\mus)');
ylabel('Amplitude');

title('Beamformed Signal Before Rate Conversion');



subplot(2,1,2);

plot(t_out(1:150)*1e6,...
    real(beamformed_out(1:150)));

grid on;

xlabel('Time (\mus)');
ylabel('Amplitude');

title('Beamformed Signal After Rate Conversion');
```

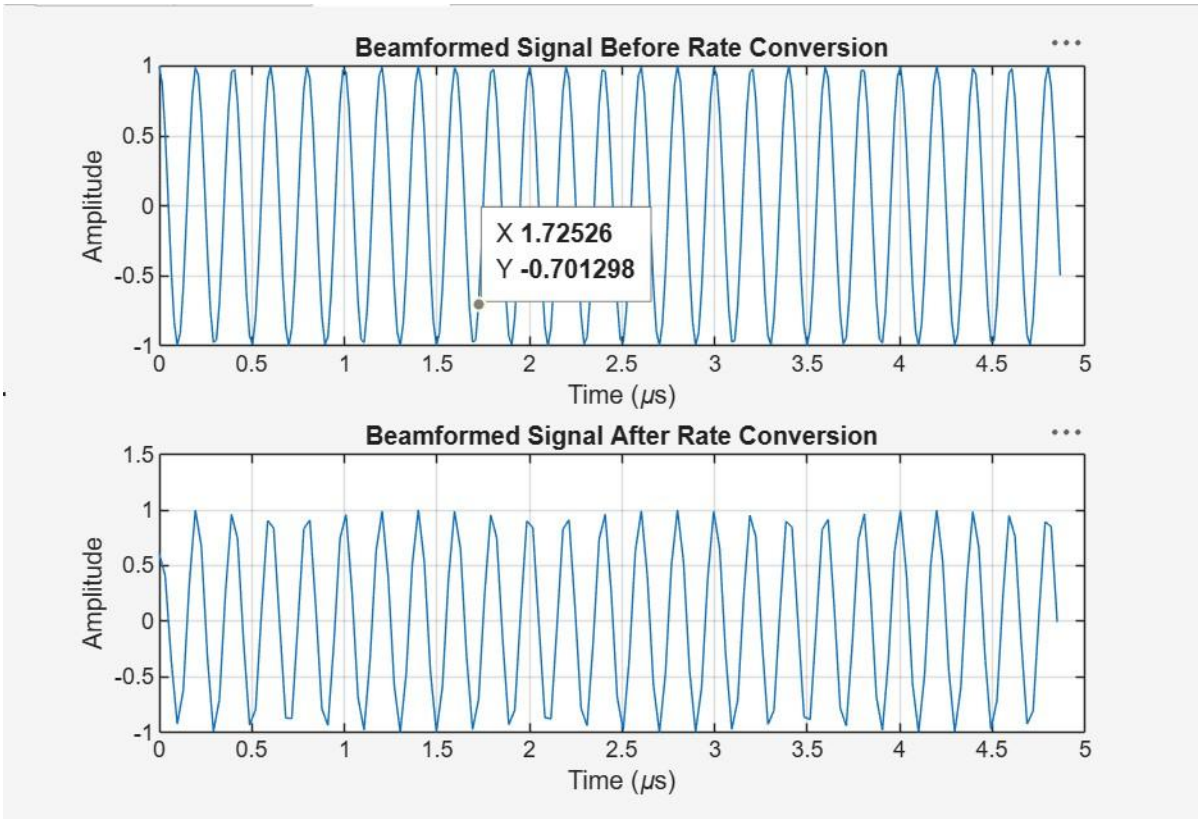
```
%% ========================================================
%  TASK 6: PERFORMANCE SUMMARY
% ========================================================

fprintf('\n=========================================\n');
fprintf('PERFORMANCE SUMMARY\n');
fprintf('=========================================\n');

fprintf('Input Sampling Rate  : %.2f MHz\n',...
   Fs_in/1e6);

fprintf('Output Sampling Rate : %.2f MHz\n',...
   Fs_out/1e6);

fprintf('Rate Conversion      : Decimation by %d\n',...
   M);

fprintf('Maximum Phase Error  : %.6f degrees\n',...
   max(abs(phase_error)));

fprintf('Amplitude Distortion : %.6f %%\n',...
   amplitude_distortion_percent);

fprintf('Beamforming Gain     : %.3f dB\n',...
   beamforming_gain);

fprintf('=========================================\n');
```

Output:

```
----------------------------------------
AMPLITUDE ANALYSIS
----------------------------------------
Amplitude before = 1.000000
Amplitude after  = 1.000925
Amplitude distortion = 0.092517 %

----------------------------------------
BEAMFORMING INTEGRITY
----------------------------------------
Input antenna power = 1.001851
Beamformed power = 1.001851
Beamforming gain = 0.000 dB

========================================
PERFORMANCE SUMMARY
========================================
Input Sampling Rate  : 61.44 MHz
Output Sampling Rate : 30.72 MHz
Rate Conversion      : Decimation by 2
Maximum Phase Error  : 0.000000 degrees
Amplitude Distortion : 0.092517 %
Beamforming Gain     : 0.000 dB
========================================
```


Spectrum Before and After Rate Conversion
Before Rate Conversion
After Rate Conversion
Magnitude (dB)
0
-20
-40
-60
-80
-100
-120
-20
-15
-10
-5
0
5
10
15
20
Frequency (MHz)


Phase Alignment Before and After Rate Conversion
Before Rate Conversion
After Rate Conversion
Relative Phase (Degrees)
100
90
80
70
60
50
40
30
20
10
0
1
1.5
2
2.5
3
3.5
4
Antenna Element



Result:

The beamformed signal was successfully converted from 61.44 MHz to 30.72 MHz using decimation by 2. The simulation showed minimal amplitude distortion, preserved phase alignment, and maintained the desired spectral and beamforming characteristics after rate conversion.